

PySUS — Introdução da Apresentação

Antes da gente entender o PySUS, primeiro precisamos entender o problema que ele resolve. E esse problema começa com dois formatos antigos do DATASUS: DBF e DBC.

O DBF é um formato muito antigo de banco de dados, criado lá nas décadas de 80 e 90 junto de sistemas como dBase e FoxPro. Ele funciona basicamente como uma tabela, parecido com um Excel antigo. Por exemplo: nome, idade, cidade, sexo, óbito, internação... tudo isso podia ser armazenado em arquivos `.dbf`.

Esse formato foi muito utilizado por governos, sistemas legados, saúde pública, cartórios e vários sistemas antigos do SUS. O DATASUS usa esse modelo há décadas.

Mas existe um segundo formato, que é o DBC. E aqui começa o verdadeiro problema.

O DBC não é exatamente outro banco de dados. Na prática, ele é um arquivo DBF comprimido usando um algoritmo proprietário chamado *blasius*. Então funciona mais ou menos assim: o DBF é o arquivo original, e o DBC é esse mesmo arquivo compactado.

O governo fazia isso porque os dados do SUS são gigantescos. Estamos falando de milhões de registros de nascimentos, óbitos, dengue, internações, vacinas e notificações de doenças. Então a compressão economizava armazenamento e tráfego de internet.

Só que surge um problema enorme: esse algoritmo *blasius* não é um padrão moderno como ZIP, RAR ou gzip. Ou seja, o Windows não abre normalmente, o Excel não entende, o pandas não consegue ler e o Python puro também não consegue trabalhar diretamente com esses arquivos.

Historicamente, para acessar os dados do DATASUS era necessário usar ferramentas específicas, como TabWin, programas antigos de conversão e vários processos manuais. O fluxo normalmente era: baixar o arquivo DBC, descompactar, converter para DBF, depois converter para CSV e só então abrir no Excel ou analisar em Python. Era uma trabalhadeira absurda.

E é exatamente aqui que entra a grande importância do PySUS.

O PySUS automatiza todo esse processo. Com apenas uma linha de código:

```
df = sinan(disease='deng', year=2024)
```

a biblioteca baixa os arquivos, descompacta, converte, transforma em formatos modernos e já entrega tudo em um DataFrame pandas pronto para análise.

Ou seja: o PySUS funciona como uma ponte entre sistemas legados antigos do governo e o ecossistema moderno de Ciência de Dados em Python.

E isso é muito importante conceitualmente, porque boa parte da infraestrutura pública brasileira ainda funciona com tecnologias antigas, como COBOL, FTP, DBF e formatos proprietários. Então bibliotecas como o PySUS praticamente fazem uma modernização da informação pública brasileira.

Outro detalhe importante é que o PySUS converte esses dados para Parquet, que é um formato moderno, colunar e extremamente rápido para Big Data. Enquanto o DBF é antigo e lento, o Parquet é otimizado para análise de grandes volumes de dados.

Então, resumindo: uma das grandes genialidades do PySUS é justamente pegar uma infraestrutura de dados dos anos 90 e transformar isso em algo compatível com o ecossistema moderno de Data Science.

Quem criou o PySUS?

Agora que entendemos o problema que o PySUS resolve, vale falar rapidamente sobre quem criou a biblioteca.

O PySUS surgiu dentro do contexto do projeto InfoDengue e posteriormente AlertaDengue, envolvendo pesquisadores da Fundação Getúlio Vargas e da Fiocruz.

Os autores citados oficialmente na publicação do projeto são:

- Flávio Codeço Coelho — principal criador e coordenador do projeto
- Bernardo Chrispim Baron
- Gabriel Machado de Castro Fonseca
- Pedro Reck
- Daniela Palumbo

O projeto é open source e está disponível no GitHub, permitindo que pesquisadores e desenvolvedores contribuam continuamente para a evolução da biblioteca.

Tutorial Prático — Utilizando o PySUS

Agora vamos entrar na parte prática da biblioteca e entender como utilizar o PySUS no Python. A primeira etapa é a instalação da biblioteca. O PySUS pode ser instalado utilizando o pip, que é o gerenciador de pacotes padrão do Python. No terminal, basta executar: `pip install pysus`. Caso a biblioteca já esteja instalada e o objetivo seja atualizar para a versão mais recente, utiliza-se: `pip install --upgrade pysus`. Depois da instalação, é possível verificar se tudo funcionou corretamente executando o seguinte bloco em Python:

Python

```
import pysus

print(pysus.__version__)
```

Se a instalação estiver correta, o Python irá mostrar a versão instalada da biblioteca.

É importante entender que o PySUS não funciona sozinho. Ele depende de várias outras bibliotecas do ecossistema Python para conseguir manipular, baixar e analisar os dados do DATASUS. Entre as principais dependências utilizadas pelo PySUS estão: **pandas** (manipulação de tabelas e DataFrames), **pyarrow** (leitura e escrita do formato Parquet), **duckdb** (consultas extremamente rápidas em grandes volumes de dados), **aiohttp** (downloads assíncronos pela internet), **numpy** (processamento numérico) e **fsspec** (acesso a sistemas de arquivos e armazenamento remoto). Ou seja: quando instalamos o PySUS, o pip também instala automaticamente todas essas dependências necessárias para o funcionamento da biblioteca. Outro detalhe importante é que algumas versões do Windows podem apresentar dificuldades com bibliotecas internas como libffi. Por isso, em alguns casos, a própria documentação recomenda o uso de Docker. Mesmo assim, na maioria dos computadores atuais, a instalação via `pip install pysus` já funciona normalmente.

Importando o PySUS

Depois da instalação, o próximo passo é importar as funções da biblioteca. Em Python, o comando `import` serve para carregar bibliotecas na memória do programa. No caso do PySUS, podemos importar diretamente funções específicas para acessar diferentes bases do DATASUS. Exemplo: `from pysus.api import sinan`. Aqui estamos importando a função `sinan()`, responsável por acessar dados do Sistema de Informação de Agravos de Notificação. Também existem outras funções importantes dentro da biblioteca, que podem ser importadas da seguinte maneira: `from pysus.api import sinasc, sim, sih, sia, pni, ibge, cnes, ciha`. Cada uma delas acessa uma base diferente do SUS, por exemplo: `sinasc()` acessa dados de nascimentos; `sim()` gerencia dados de mortalidade; `sih()` monitora internações hospitalares; `pni()` lida com vacinação; e `cnes()` mapeia os estabelecimentos de saúde.

Baixando Dados com o PySUS

Agora vem a parte mais importante: baixar os dados. Um dos exemplos mais utilizados é o acesso aos dados de dengue do SINAN. O código abaixo baixa os casos de dengue registrados no ano de 2024:

Python

```
from pysus.api import sinan

df = sinan(disease='deng', year=2024)
```

Aqui acontece uma coisa extremamente importante internamente. Quando executamos essa linha, o PySUS realiza uma série de tarefas de forma transparente: localiza os arquivos no DATASUS, baixa os arquivos .dbc, descompacta-os automaticamente, converte os dados, transforma-os em um formato moderno e entrega tudo estruturado em um DataFrame pandas. Tudo isso acontece automaticamente, e o resultado final é armazenado na variável `df`. E aqui entra um conceito importante: `df` significa “DataFrame”. O DataFrame é a principal estrutura de dados do pandas e funciona como uma tabela inteligente, semelhante a uma planilha do Excel, porém muito mais poderosa para análise de dados.

Explorando os Dados

Depois do download, podemos começar a explorar os dados. Por exemplo, o comando `print(df.shape)` exibe o atributo `.shape`, que mostra o tamanho do DataFrame em termos de quantidade de linhas e quantidade de colunas. Se o retorno for algo como `(150423, 87)`, isso significa que a tabela possui 150.423 registros e 87 colunas de informação. Também podemos visualizar as primeiras linhas da tabela utilizando `print(df.head())`. O método `.head()` mostra os primeiros registros do DataFrame, permitindo uma inspeção rápida dos dados estruturados. Outra possibilidade para entender quais informações existem dentro da base do DATASUS é visualizar os nomes de todas as colunas disponíveis por meio do comando `print(df.columns)`.

Analisando os Dados

Depois de carregar os dados, podemos começar as análises. Um bom exemplo é quantificar os atendimentos por período, calculando a quantidade de casos por mês:

Python

```
import pandas as pd

df['MES'] = pd.to_datetime(df['DT_NOTIFIC']).dt.month

casos_mes = df.groupby('MES').size()

print(casos_mes)
```

Aqui existem vários conceitos importantes. A coluna `DT_NOTIFIC` representa a data de notificação da doença. O comando `pd.to_datetime()` transforma texto em data real, permitindo manipulações temporais. Depois, o sufixo `.dt.month` extrai apenas o número correspondente ao mês daquela data específica. Em seguida, a função `groupby('MES')` agrupa todos os registros com base no mês extraído e, finalmente, o método `size()` conta quantos casos existem em cada grupo. Ou seja: estamos criando uma análise temporal completa dos casos de dengue.

Também podemos realizar filtrações específicas, como isolar os casos graves de dengue utilizando a seguinte linha de código: `graves = df[df['CLASSI_FIN'] == '12']`. Aqui o código está filtrando e selecionando apenas os registros cuja classificação final (`CLASSI_FIN`) é igual a '12', que é o código padrão para representar dengue grave. Depois de realizar essa separação, podemos calcular a porcentagem desses casos em relação ao total da base: `pct = len(graves) / len(df) * 100`. A função `len()` conta quantos registros existem em um conjunto de dados. Portanto, o cálculo divide a quantidade de casos graves pela quantidade total de casos e multiplica o resultado por 100 para gerar a porcentagem exata.

Geração de Gráficos

Depois de realizar as análises estruturais, podemos gerar visualizações gráficas para facilitar a interpretação dos resultados. Exemplo de geração de gráfico:

Python

```
import matplotlib.pyplot as plt

casos_mes.plot(kind='bar')

plt.title('Casos de Dengue por Mês')

plt.show()
```

Nesse bloco, o `matplotlib` é utilizado para construir um gráfico de barras. O método `plot(kind='bar')` transforma os dados agregados da variável anterior diretamente no formato visual de barras, enquanto a função `plt.title()` define o título que será exibido no topo do gráfico e `plt.show()` renderiza e exibe a visualização na tela do usuário.

Acessando Outras Doenças no PySUS

O PySUS não trabalha apenas com dengue. A biblioteca permite acessar diversas doenças notificáveis do SINAN utilizando apenas a troca do código da doença no parâmetro `disease`.

Por exemplo:

```
from pysus.api import sinan
```

Dengue:

```
df = sinan(disease='deng', year=2024)
```

Zika vírus:

```
df = sinan(disease='zika', year=2024)
```

Chikungunya:

```
df = sinan(disease='chik', year=2024)
```

Tuberculose:

```
df = sinan(disease='TUBE', year=2024)
```

Leishmaniose Visceral:

```
df = sinan(disease='LEIV', year=2024)
```

Ou seja: a estrutura do código praticamente não muda. O que muda é apenas o identificador da doença.

Isso mostra uma característica muito importante do PySUS: a padronização da API.

Conclusão do Tutorial

O PySUS transforma um processo extremamente complexo de acesso aos dados públicos brasileiros em algo simples e automatizado. O que antes exigia ferramentas antigas, múltiplas conversões e muito trabalho manual, agora pode ser realizado com poucas linhas de código em Python. Além disso, a biblioteca aproxima o ecossistema moderno de Ciência de Dados dos sistemas públicos brasileiros, permitindo pesquisas, análises epidemiológicas e estudos científicos de maneira muito mais acessível.